

In an `ArrayList<Player>`, how can I validate shirt number uniqueness for both add and update operations? During an update, the current player should be allowed to keep the same shirt number without being treated as a duplicate.

Thought for a few seconds >

Use one shared validation method, then **exclude the player currently being edited** during update.

```
</> Java
public boolean isShirtNumberTaken(int shirtNumber, String excludePlayerId) {
    for (Player player : playerList) {
        boolean sameShirtNumber = player.getShirtNumber() == shirtNumber;
        boolean isAnotherPlayer = !player.getPlayerId().equals(excludePlayerId);

        if (sameShirtNumber && isAnotherPlayer) {
            return true;
        }
    }
    return false;
}
```

When adding a player

Pass `null` because there is no current player to exclude.

```
</> Java
if (isShirtNumberTaken(shirtNumber, null)) {
    System.out.println("This shirt number is already assigned.");
    return;
}

playerList.add(newPlayer);
```

To avoid a `NullPointerException`, slightly adjust the method:

```
// Java

public boolean isShirtNumberTaken(int shirtNumber, String excludePlayerId) {
    for (Player player : playerList) {
        boolean sameShirtNumber = player.getShirtNumber() == shirtNumber;

        boolean isAnotherPlayer = excludePlayerId == null
            || !player.getPlayerId().equals(excludePlayerId);

        if (sameShirtNumber && isAnotherPlayer) {
            return true;
        }
    }
    return false;
}
```

When updating a player

Pass that player's own ID:

```
// Java

Player player = findPlayerById(playerId);

if (isShirtNumberTaken(newShirtNumber, player.getPlayerId())) {
    System.out.println("This shirt number belongs to another player.");
    return;
}

player.setShirtNumber(newShirtNumber);
System.out.println("Player updated successfully.");
```

So, for example, Player `P01` can keep shirt number `10`, because the validation ignores `P01`; but assigning `10` to Player `P02` will be rejected.

```
/* Returns true when another player is already using this shirt number. */
private boolean isShirtNumberTaken(int shirtNumber, String currentPlayerId) {
    for (Player player : players) {
        boolean sameShirtNumber = player.getShirtNumber() == shirtNumber;
        boolean isAnotherPlayer = currentPlayerId == null
            || !player.getPlayerId().equals(currentPlayerId);

        if (sameShirtNumber && isAnotherPlayer) {
            return true;
        }
    }
    return false;
}

// Ignore the current player so keeping the same shirt number is allowed.
if (isShirtNumberTaken(player.getShirtNumber(), player.getPlayerId())) {
    System.out.println("This shirt number is already assigned to another player.");
    int newShirtNumber = inputAvailableShirtNumber(
        sc,
        "Enter another shirt number: ",
        player.getPlayerId()
    );
    player.setShirtNumber(newShirtNumber);
}
```